



AGENTIC MANDI TESTERS

AI Powered API Documentation and Testing Portal

Turning raw API definitions into documentation, sample requests, test cases, and actionable error guidance.

TEAM

Raghav Satija (Roll No. 311)

Jnanesha V (Roll No. 383)

Thanush M (Roll No. 315)

Problem Statement & Real-World Impact

Build a working solution for an AI-Powered API Documentation and Testing Portal that takes relevant input data, processes it through a clear workflow, and produces an actionable output for real users or decision-makers. The solution should not stop at code execution — it should help a user understand what action to take next.



Docs go stale or never get written

Teams skip writing developer docs under deadline pressure, leaving new endpoints undocumented.



Edge-case tests are repetitive

Manually enumerating auth/validation/not-found cases for every route is tedious and error-prone.



Raw errors don't say what to fix

A bare '422 Unprocessable Entity' tells a developer nothing actionable about the next step.

Real-world impact: faster onboarding for new developers, fewer support tickets caused by unclear errors, and a CI-ready test suite generated in minutes instead of days.

Dataset & Reference Material

data/endpoints.json — 100 realistic endpoint records across 20 resources

Field	Meaning
method	HTTP verb: GET, POST, PUT, DELETE
path	Route, incl. path params e.g. /orders/{order_id}
description	Short summary of intent
request_body	Expected input fields & types
response_body	Returned fields & types
auth_required	Boolean — needs Authorization header
error_codes	List of possible HTTP error statuses

20 RESOURCES COVERED

users · products · orders · payments · invoices · categories · reviews · carts ·
addresses · notifications · auth · wishlists · coupons · shipments · suppliers ·
warehouses · tickets · messages · subscriptions · reports

100

Endpoint records

20

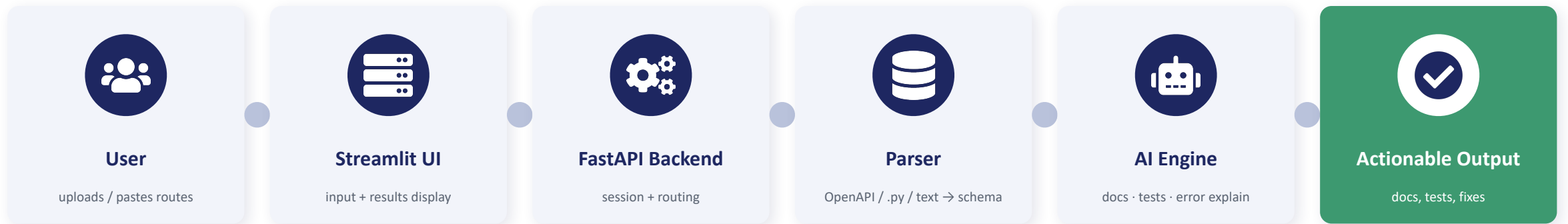
Resources

5

CRUD ops / resource

Plus an error_code_reference table (400–500) shared across all endpoints, and support for uploading real OpenAPI/Swagger specs or FastAPI/Flask source files instead of the starter set.

System Workflow



PROCESSING PATH

- Upload OpenAPI / .py / paste text
- Parser normalizes to a common schema
- AI Engine generates docs + tests per endpoint
- Results rendered in Streamlit, ready to copy into code/CI

LIVE TESTING PATH

- User enters a real base URL + headers/body
- Backend calls the live API via httpx
- On failure, AI explains cause + fix in plain English
- User knows exactly what to change next

AI / Agent Innovation — Where & Why



1. Documentation Generator

Reads the sparse schema (method, path, body) and writes human prose: purpose, plain-English field explanations, sample request/response, usage notes & gotchas.



2. Test-Case Generator

Infers which edge cases a schema implies — missing auth, invalid payload, not-found id — and emits runnable pytest snippets, not just descriptions.



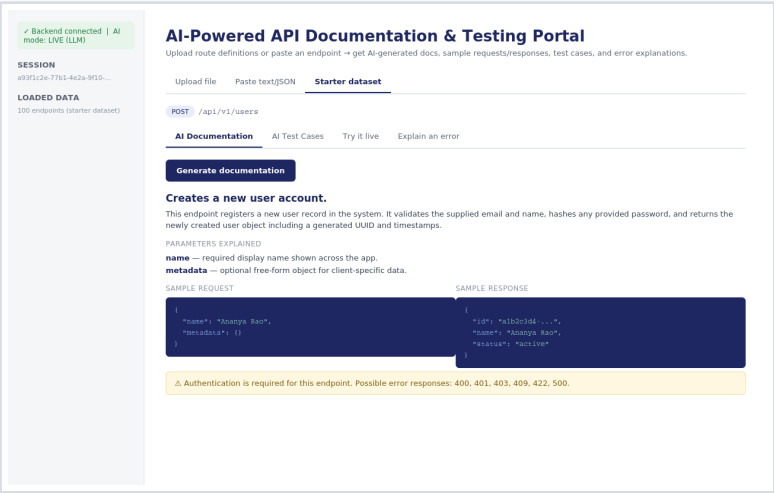
3. Error Explainer

Given a real failed response (status + body), explains the likely cause, plain-English meaning, and a concrete suggested fix — the actionable output.

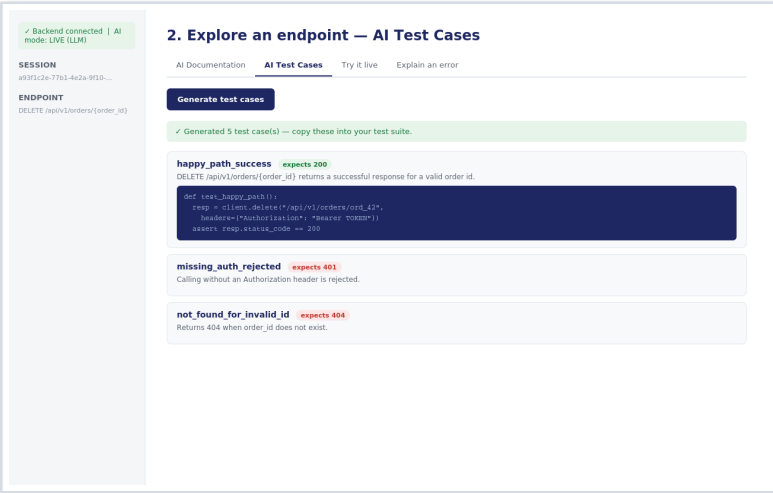


Why this matters: any OpenAI-compatible LLM endpoint (OpenAI or NVIDIA NIM) plugs in unchanged. Every AI feature also has a deterministic template fallback, so the portal runs end-to-end with zero API key — reliable for demos with no internet, and a clear illustration that the AI adds reasoning on top of a working system, not a dependency it can't function without.

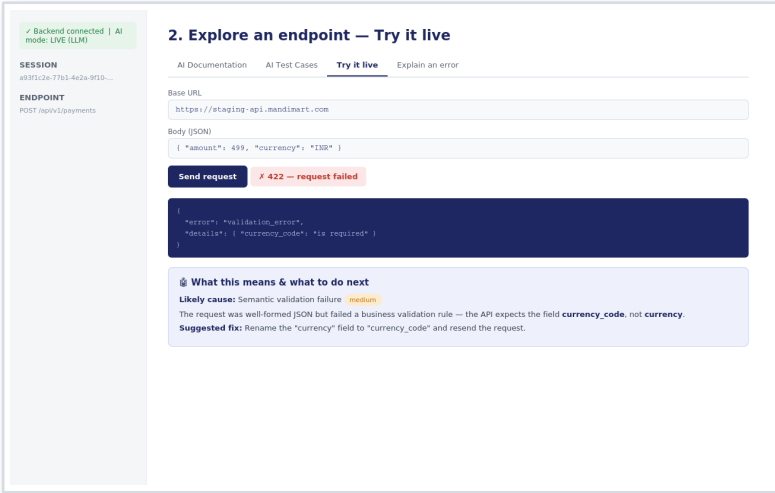
Demo Screenshots



AI-generated documentation



AI-generated test cases



Live call + AI error explanation

All three screens share one Streamlit session: load endpoints once, then switch tabs per endpoint to generate docs, tests, run a live call, or decode any error.

- 1

Load endpoints
- 2

Pick one
- 3

Generate / Test / Try it
- 4

Act on the result

SAMPLE OUTPUT

Results — From One Endpoint Definition

INPUT

POST /api/v1/payments

```
{
  "method": "POST",
  "path": "/api/v1/payments",
  "request_body": {"amount": "number", "currency_code":
"string"},
  "auth_required": true,
  "error_codes": [400,401,403,409,422,500]
}
```

OUTPUT (AI-GENERATED)

- Plain-English doc: “Creates a payment for the supplied amount and currency.”
- 5 test cases: happy path, missing auth, invalid payload, not-found, duplicate
- Runnable pytest snippet per test case
- Error fix: “rename 'currency' to 'currency_code'” on a live 422

PORTAL-WIDE RESULTS

100

endpoints documented from the starter dataset in one batch-ready run

~5

test cases generated per endpoint, covering auth, validation & not-found

3

input formats supported: OpenAPI/Swagger, FastAPI/Flask source, freeform text

0

API keys required to run the full demo — template fallback covers every feature

Limitations & Responsible Use



Known Limitations

In-memory session storage

Loaded endpoints reset when the backend restarts — fine for a demo, not for production multi-user use.

Heuristic Python parser

Regex-based FastAPI/Flask route extraction won't catch every code style or dynamic route registration.

AI can be wrong

Generated docs, tests and fixes are a strong starting draft, not a substitute for human review before shipping.



Responsible Use Practices

No secrets are stored

Auth tokens entered in 'Try it live' are used only for that single request, never persisted.

Live calls are user-initiated only

The portal never calls a real API unless the user explicitly enters a base URL and clicks send.

Always review before production

Generated test cases and docs should be reviewed by a developer before merging into CI or shipping to users.

Future Improvements



Persistent storage

Move from in-memory sessions to a real database so teams can share and revisit projects.



Auto-run generated tests

Execute the generated pytest suite directly against a staging API and show pass/fail inline.



Multi-user collaboration

Accounts, shared workspaces, and comment threads on individual endpoints.



CI/CD integration

Export generated tests straight into a GitHub Actions workflow on every PR.



Agentic regression watch

An agent that periodically re-tests live endpoints and flags breaking changes automatically.



Versioned doc history

Track how an endpoint's docs/behavior changed release over release.

CONCLUSION

From Raw Endpoints to Action — In One Portal

The portal takes whatever API definition a developer already has — OpenAPI, source code, or a quick paste — and turns it into documentation, runnable tests, and concrete error fixes. The AI layer doesn't just generate text; it closes the loop by telling the user exactly what to do next, which is what makes the output actionable rather than just generated.

- ✓ Working end-to-end prototype
- ✓ Clear AI/agent justification
- ✓ Runs with or without an API key
- ✓ Real, testable starter dataset

Thank you.

AGENTIC MANDI TESTERS

Raghav Satija

Roll No. 311

Jnanesha V

Roll No. 383

Thanush M

Roll No. 315